Input → Backbone

before making our Backbone, need to define labels for the backbone.

Feature	KITTI Language (The "text")	YOLO Language (what AI works with)
Class	written words (eg. "car", "pedestrian)	Numbers (eg. 0, 1)
Coordinates	Pixels (x_{min}, y_{min}, x_{max}, y_{max})	Percentages (x_{center}, y_{center}, width, height)
Extra Info	15 columns (Rotation, Truncation, etc)	5 columns (Box and the class)

inside label_2_first 100.xxxxxxtxt

ex.) Car 0.00 -1.57 599.41 156.40 629.75 189.25 ..... -1.56

Column 0 1 2 3 4 5 6 15

what we need : 5 columns for YOLOv8.

1.) class 2~5.) x_{min}, y_{min}, x_{max}, y_{max}

⇓

[Class ID] [x_center] [y_center] [width] [height]

Step A.) Width & Height

$$width = x_{max} - x_{min}$$

$$height = y_{max} - y_{min}$$

Step B.) Center point (x, y)

$$x_{center} = x_{min} + \frac{width}{2}$$

$$y_{center} = y_{min} + \frac{height}{2}$$

Step C.) Normalize x_{center} y_{center} W H

Backbone

- Feature Pyramid
  - Top Layer (the input)
    - image is big & clear, but the computer doesn't understand it.

    ⋮

  - Bottom Layer (the output)
    - image is tiny & blurry, but the computer extracted "meaning"

How we do it?

1.) "Scan and Shrink" process : Convolutional layer.

why?

- To find the pattern : A Convolutional layer acts like magnifying glass with specific filter (eg. vertical lines). As it slides over the image, it highlights everywhere it sees a vertical line.

- To summarize info : We don't want the AI to look at all 640 x 640 pixels forever - too much data! We use a Stride (a jump). If the filter jumps 2 pixels at a time, the resulting is half the size.

```
import torch
import torch.nn as nn

Scanner = nn.Conv2d( in_channel = 3, out_channel = 16, kernel_size = 3, stride = 2,
                                                                        padding = 1)
```

① ② ③ ④ ⑤

① in_channel = 3 (The input)

- computer doesn't see a "picture", it sees three layers of numbers (Tensors)
  - Layer 1: All the Red values
  - Layer 2: All the Green values
  - Layer 3: All the Blue values.

② out_channel = 16 (The pattern maps)

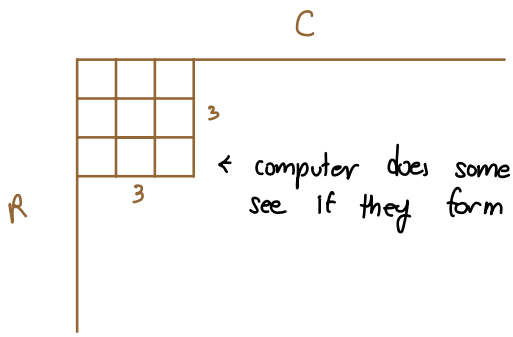
- Creates 16 different glasses at once.
  - Glass #1 might be looking for Vertical Lines
  - Glass #2 " " " Horizontal Lines
  - Glass #3 " " " Circular Curves (like wheels)

when the code finishes, it gives you 16 "Pattern Maps"

- Map #1 will be bright in places where there are vertical lines
- Map #3 " " " " " " " " wheels

③ kernel_size = 3 (The magnifying glass)

- 3 x 3 glass board.



← computer does some math (multiplication) on these 9pixels to see if they form an "edge" or a "curve"

- 3 x 3 square is the kernel (filter)

④ Stride = 2 (The Jump)

- Stride = Jump (remember!?)

∴ stride = 2 means that your kernel would jump 2 pixels at a time. to the right allowing to "summarize" the data.

⑤ padding = 1 (The border)

- when you put a 3x3 Glass on the photo, the glass "hangs off" the side, since there are no pixels there.

- padding = 1 tells the computer to imagine a 1-pixel thick border of zeros around the photo. This ensures the magnifying glass can reach the edge of the image without "falling off".

out_channels = 16 ?

$$\text{Output}(j) = \text{Bias}(j) + \sum_{k=1}^{3} \text{Filter}(j,k) * \text{Input}(k)$$

j = The specific output map (1~16)
k = The input color channel (Red, Green, Blue)
* = sliding multiplication. (cross-correlation)

Recall : $\sigma(W \cdot A + b)$ "global" formula for a <u>single</u> neuron.

1. Comparison

Part	$\sigma(W \cdot A + b)$	$\text{Output}(j) = \text{Bias}(j) + \sum_{k=1}^{3} \text{Filter}(j,k) * \text{Input}(k)$
Weights (W)	Simple list of weights	3x3 kernel (magnifying glass)
Input (A)	Input value	The pixels inside 3x3 window
Bias (b)	A single number added at the end	A single number added to the Pattern map
Activation	Sigmoid (0~1)	SiLU (modern version of Sigmoid)

2. "Sliding" difference.

On our formula we use (cross-correlation) *

- $W \cdot A + b$ on pixels 1-9
- $W \cdot A + b$ on pixels 2-10
- Continue for the entire image.

3. SiLU vs Sigmoid.

$$\text{sigmoid} = \sigma(x) = \frac{1}{1+e^{-x}}$$

↳ problem: If the x gets too big, the "slope" of the curve becomes almost flat, and the Gradient Descent stops working.
(Vanishing Gradient Descent)

YOLO uses SiLU (sigmoid Linear Unit).

- formula: $f(x) = x \cdot \sigma(x)$
- why: Allows some negative numbers to pass through, which helps "Gradient Descent" to stay "alive"

1.) In a single neuron formula: $(W \cdot A + b)$, W is one number.
In our Conv2d layer: Weights are the numbers inside 3x3 kernel glasses.

we have 16 filters, each looking at 3 channels (R, G, B) with 3x3 size (kernel).

$16 \times 3 \times 3 \times 3 = 432$ weights.

When we initialize nn.Conv2d (out_channels) = 16, PyTorch creates 432 random numbers.
these are balls on a complex 432-dimensional Hill.

2.) The "Cost" Function

The "Bad": 16 Pattern Maps produced by our random weights. (look like gray static)

The "Need": The KITTI Labels

- If the Pattern Maps are totally blank, where the Car is supposed to be, the Cost Function (a.k.a the loss function) will be very high.

3) "Nudging"

1. Forward Pass: You slide 16 filters over the image
2. Loss: You compare the output to the Car Label.
3. Backpropagation (∇C): PyTorch calculates the partial derivative of the cost with repect to every single one of those 432 numbers.
"If I make the top-left pixel of Filter #5 a little more positive, does

the Car Detection' get any better?"

4. Nudge : takes negative gradient and updates 432 numbers.